

Brief Project Report

Plant Disease Detection using CNN and Transfer Learning

Bubai Sou

25MA60R14

M.Tech CSDP

Department of Mathematics

IIT Kharagpur

Repository: <https://github.com/Bubai2026/plantDisease/>

Kaggle dataset: <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset/>

Objectives are:

- To build a complete end-to-end image classification pipeline for detecting plant diseases from leaf images.
- To train and compare different CNN models, some made from scratch and some using transfer learning.
- To find the model which gives the best performance by comparing the main evaluation metrics.
- To use Grad-CAM visualizations so I can check if the model is actually looking at the diseased parts of the leaf, and not some random background feature.

Dataset:

For my project I used the PlantVillage dataset, which I took from Kaggle. It contains around 54,000 leaf images and 38 classes of healthy and diseased leaves from 14 different crop species. The dataset has color, grayscale and segmented versions of the images. I used the color images for this project.

Class Labels Table(class_indices.json)

Index	Crop	Class Label (Condition)
0	Apple	Apple scab
1	Apple	Black rot
2	Apple	Cedar apple rust
3	Apple	Healthy
4	Blueberry	Healthy
5	Cherry (incl. sour)	Powdery mildew
6	Cherry (incl. sour)	Healthy
7	Corn (maize)	Cercospora leaf spot / Gray leaf spot

8	Corn (maize)	Common rust
9	Corn (maize)	Northern leaf blight
10	Corn (maize)	Healthy
11	Grape	Black rot
12	Grape	Esca (Black Measles)
13	Grape	Leaf blight (Isariopsis leaf spot)
14	Grape	Healthy
15	Orange	Huanglongbing (Citrus greening)
16	Peach	Bacterial spot
17	Peach	Healthy
18	Pepper, bell	Bacterial spot
19	Pepper, bell	Healthy
20	Potato	Early blight
21	Potato	Late blight
22	Potato	Healthy
23	Raspberry	Healthy
24	Soybean	Healthy
25	Squash	Powdery mildew
26	Strawberry	Leaf scorch
27	Strawberry	Healthy
28	Tomato	Bacterial spot
29	Tomato	Early blight
30	Tomato	Late blight
31	Tomato	Leaf mold
32	Tomato	Septoria leaf spot
33	Tomato	Spider mites (Two-spotted spider mite)
34	Tomato	Target spot
35	Tomato	Tomato Yellow Leaf Curl Virus
36	Tomato	Tomato mosaic virus
37	Tomato	Healthy

Methodology:

I did the project in a few steps. First I made a simple CNN as a baseline, then I improved it. After that I tried some transfer learning models and finally I used fine tuning on the best model. Below is the whole methodology:

1. Data Acquisition and Preparation:

I downloaded the PlantVillage dataset using the Kaggle API inside my notebook. After downloading it, I arranged the images in class wise folders so that data loaders can use them. I also did some basic EDA to check the class distribution and looked at sample images from

different crops and diseases. This helped me check that the images and labels looked correct before training.

2. Preprocessing and Augmentation:

I resized all the images to 160×160×3 and changed the pixel values to the [0, 1] range. For training, I used ImageDataGenerator with augmentation like rotation of 20° anticlockwise or clockwise, zoom of 0.2, width and height shifts of 0.2 and horizontal flipping. I did this so the model can see slightly different versions of the leaves and hopefully reduce overfitting.

I only rescaled the validation images and did not augment them, so that my validation data stays more like the original images. I loaded the data using `flow_from_directory` with an 80/20 train-validation split and categorical labels.

3. Baseline CNN:

I first build a simple 3-block CNN from scratch so I could use it as a reference model. It had three Conv2D and MaxPooling blocks with 32, 64 and 128 filters. After this I used a Flatten layer, Dense layer with 256 neurons and ReLU, Dropout of 0.5 and finally a Dense 38 class Softmax output. I used Adam optimizer with learning rate 10^{-3} and categorical cross-entropy as the loss. I also used EarlyStopping with patience 3 and validation loss monitoring, and ReduceLROnPlateau with factor 0.2 and patience 2.

4. Improved CNN:

I then improved my baseline CNN by putting Batch Normalization after each convolution layer, adding Dropout of 0.3 after every pooling block, and using L2 kernel regularization of 10^{-4} on the Conv2D layers. My main aim was to reduce overfitting and make the training more stable. But I kept the optimizer, loss and callbacks same as the baseline so my comparison would be fair.

5. Transfer Learning:

After my CNN experiments, I tried transfer learning using three ImageNet pretrained models: MobileNetV2, ResNet50 and EfficientNetB0. I first used these models as frozen feature extractors.

For each backbone, I applied the model specific `preprocess_input` function through a Lambda layer. I loaded the pretrained base with `include_top=False` and froze its weights. Then I added a common classification head: Global Average Pooling → Dense(256, ReLU) → Dropout(0.4) → Dense(38, Softmax). I used Adam with learning rate 10^{-3} and categorical cross-entropy for all three models. I also used the same EarlyStopping and ReduceLROnPlateau callbacks for all of them.

6. Fine-Tuning:

I selected it for fine tuning , since EfficientNetB0 was giving me the best result. I made the base model trainable but only unfroze the top 30 layers. I kept the earlier layers frozen because they contain more general features, while the deeper layers can learn features more specific to plant

diseases. I compiled the model again with a much smaller learning rate of 10^{-5} so that the pretrained weights don't get changed too much. Then I trained it for 5 more epochs with the same callbacks.

7. Evaluation and Interpretability:

I evaluated all the models, starting from the baseline CNN to the fine tuned EfficientNetB0, on the held-out validation set. I calculated Accuracy, Precision, Recall and F1-Score and also used confusion matrices for comparing the models.

I applied Grad-CAM on my final fine-tuned model. It produced heatmaps showing which areas of the leaf was having the most influence on the prediction.

Results:

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Baseline CNN	72.4	71.8	72.4	72.1
Improved CNN	82.6	82.3	82.6	82.4
MobileNetV2 (Transfer)	93.1	92.8	93.1	92.9
ResNet50 (Transfer)	94.5	94.2	94.5	94.3
EfficientNetB0 (Transfer)	96.7	96.5	96.7	96.6
EfficientNetB0 (Fine-Tuned)	98.9	98.8	98.9	98.8

Main Result: My fine-tuned EfficientNetB0 gave the best result, with 98.9% accuracy and 98.8% F1-score.

Grad-CAM Interpretability:

I used Grad-CAM with my fine-tuned EfficientNetB0 model to see which parts of the image was affecting the prediction. The heatmaps showed strong attention around the disease affected regions of the leaves, while comparatively less attention was seen on the background. This suggest that my model is mostly using disease related visual patterns instead of just the background or general leaf shape.

Conclusion:

In this project I made a complete end-to-end deep learning pipeline for plant disease classification. I started with a basic CNN and then moved to an improved CNN, transfer learning and finally fine tuning EfficientNetB0.

My final fine-tuned model reached 98.9% accuracy with 98.8% F1-score. Grad-CAM also gave me a visual way to check that the model was focusing on disease related parts of the leaf.

Overall, I feel this project shows a practical way of using deep learning for automatic plant disease detection, and it can be further developed for real world use.